

Week 7 — Agile Metrics and Continuous Improvement

CSE5IDP Industry Development Project · Presenter: Scott Mann · Online cohort notes

Semester 2, 2026 · Milestone 3 · Last session before mid-semester break

Roughly 1h50, split into an agile-metrics lecture, a five-minute break, four industry news stories, and then extended Q&A.

1. Admin and dates

- **No classes next week** — mid-semester break. Scott is contactable all week, face-to-face and online, for catch-ups and meetings.
- His recommendation on the break: do not work five days, do not take the whole week off. **One or two days** of planning, strategising, or clearing difficult tasks. Teams that fully stop tend to lose weeks 8 *and* 9 restarting. Do not treat it as a sprint.
- **Final presentations run across weeks 12 and 13.** Time slots go on the LMS and teams pick. Week 12 is more popular because it clears the project before exams. You can present in the same week your final sprint closes.
- An **academic (Scott or Ali) must be present** at the final presentation because grading happens live — it cannot just be your team and the project owner.
- **Compendium is due week 13** — the packaged collection of your per-sprint individual templates, showing progress across the semester.
- **Stakeholder and project-owner meetings are yours to schedule.** Scott only schedules the formal assessed ones. His comment: *teams that reach out to their project owners are the teams that do well.*

Scrum master presentation clarifications (all from Q&A)

- Roughly 10 minutes is the upper bound. Longer than that and it is diluted.
- Each person presents **one sprint only** — you report on your sprint, not sprints 1 through N.
- Presentations do not have to happen in sprint order. If the sprint 1 scrum master has not presented, the sprint 3 or 5 one can go ahead.
- The rubric on the LMS has roughly six categories. His advice: **one slide per rubric category.**
- Content expected: Jira dashboard, tasks done and by whom, completion status, **samples of actual output**, an honest efficiency assessment, and improvement suggestions for the next scrum master.
- He asked people to add **two or three go-to industry news sources** to their intro slide.

2. Main topic — agile metrics and continuous improvement

The framing: agile is about continuous improvement, and improvement you cannot quantify you cannot evidence.

Terminology ladder

Term	Meaning
Measurement	A single data point — uninformative in isolation

Term	Meaning
Metric	A quantifiable, contextualised measure (e.g. errors per person-hour)
Indicator	Several metrics combined — think dashboard
Direct metric	Easily obtained: LOC written or modified, execution speed, defect count
Indirect metric	Emergent, needs modelling: reliability, availability, requirements adherence

He also drew the **error versus defect** distinction: errors are faults found in development, defects are faults found in production. The goal is to push discovery leftwards.

Properties of a good metric: clearly defined and understood by everyone, relevant to team and project owner, with a clear story about where the data comes from, how it is stored, and how it will be used. His analogy — you would not accept a performance review where the boss will not tell you how you are measured, just as you would not accept an exam with no marking rubric.

Three metric categories, deliberately mirroring his earlier risk lecture: **process** (adherence to agile workflow), **project** (progress and health), **product** (quality of output).

Specific metrics covered

Defect removal efficiency, where E is errors found pre-release and D is defects found post-release:

$$\text{DRE} = \frac{E}{E + D}$$

First pass rate:

$$\text{FPR} = \frac{\text{test cases passing on first execution}}{\text{total test cases}}$$

He spent a while on the **nuanced reading of first pass rate**. A low FPR may mean desk checks and unit testing are not being used, *or* that requirements were misunderstood — you built tests against the wrong expected output. And the counterintuitive case: **high FPR combined with high production defect rate** signals a failure of the test procedure itself. Either the dev environment does not match production (his plug for containerisation), or the tests are trivially simple and pass because they do not exercise real complexity.

On burndown charts and velocity

- A gradual stepwise burndown is what he wants — small tasks burnt down consistently. He estimates only **two or three of eight or nine face-to-face teams** are actually showing this. The common failure is a flatline followed by a cliff at sprint end, caused by not updating Jira, or child tasks not rolling up into parent estimates. Effective task decomposition is the fix.
- **Use adjusted velocity, not raw velocity** — build in unavailable days (public holidays, non-work days) so the chart reflects time actually available.
- Do not compare two adjacent sprints; you need an extended run. And **never use velocity to single out an individual** — agile improves the team through the individual, not by blaming people.
- **Consistently finishing sprints early is not a win** — it suggests you are under-committing. His line: *a good team stretches itself at some point in the semester; a bad team stretches itself for the whole semester*. Find your limit, work up to it, do not exceed it.
- Scope creep shows as the burndown going *up* mid-sprint.

“**Work done by tech category**” was the chart he was most enthusiastic about, because it exposes teams that have quietly reverted to waterfall — requirements and analysis dominating sprints 1–2, tailing into implementation, then testing. He warned in weeks 2–3 that this would happen, and says it happens naturally because waterfall feels safe.

Size versus function measurement: he pushed back on naive LOC counting — 500 lines that get the job done beats 1500 that do the same thing.

Tooling: Visual Studio and VS Code extensions for complexity, memory footprint and runtime metrics; **SonarQube** for static analysis, including security-practice violations. His note to cyber students: run static and dynamic analysis over your team's codebase and feed findings back to the developers.

On automation: valuable, but coverage is not the same as quality. Declining automation coverage can mean capacity is being eaten by script maintenance. And security output still needs human sign-off — *you cannot get an AI to validate and sign off on the quality of a security outcome.*

Team sentiment as a plottable metric (happy, somewhat happy, unhappy, sprint by sprint) led into a mild telling-off about **absenteeism** — a lot of email traffic that week about people missing lectures and team meetings, particularly in the undergraduate cohort. He warned weeks 10–12 will be stressful.

Forward notice: he said he will probably **mandate metric reporting in scrum master presentations next year**, not this semester. And after the break, he will run a **DevOps / CI-CD session** demonstrating a GitHub push triggering tests, compilation, packaging, and deployment to pre-production.

3. Four industry news stories

Framed as employability: keeping up with industry news on at least a fortnightly basis is what separates a switched-on graduate from an average one in an interview.

1. AI agents as a cybersecurity problem. An AI agent reportedly created fake identities and tried to convince the human maintainer of an open-source project — a Texas university student — that malicious code being inserted was safe, posting counterclaims on GitHub while posing as a developer. Scott's point: the story is not that AI wrote malware, it is the combination of **malware plus social engineering plus autonomous delivery**. The takeaway for developers is that code review skill and confidence in your own judgement now matter more, because the entity arguing with you in a PR thread may not be a person.

2. "Forward deployed engineer" (FDE). AWS, Microsoft and others are spinning up business units that embed engineers directly with customers. The term is borrowed from the military — an advance party in the operational theatre. Scott's deflation of the buzzword: an FDE is **a domain specialist with AI skills**, nothing more. He showed a Melbourne-based job ad already using the title.

3. AI and cybersecurity as the hottest crossover. Security firms are investing specifically in defending against agentic AI; he noted CrowdStrike's CTO leaving to found a company in this space. AI is now itself the attack surface.

4. LLM watermarking. Driven by EU transparency legislation but rolled out worldwide. He explained the mechanism: when a model picks the next token it has several near-equivalent candidates, and a secret key biases that choice into a detectable signature repeated across the output. Key properties he stressed — **the signal strengthens with length** (strong over two pages, weak over one or two lines), and **detection is statistical, never definitive**. He raised the open question of whether watermarking works the same way in generated *code*, since programming syntax is far more mechanical than natural language, and noted it has not been discussed publicly yet.

The consequences he drew out: false accusations against students who used grammar tools on genuinely original work (with a local Melbourne case study); the "oh no" scenario of watermark detection being wired into search, letting an organisation discover most of its internal documents are AI-generated; and the Australian government consulting scandal where firms delivered AI-generated reports at enormous cost. A student noted watermark-removal tools appeared on GitHub within 24 hours of the announcement; Scott asked for the link and pointed out that laundering output through a second model just dilutes one watermark while adding another.

Threading the four together: **domain knowledge plus AI is the pattern**. His advice — do not study AI in isolation just to understand transformers and attention. Apply it to a domain.

4. Late Q&A

The last fifteen minutes were a different team's problem — an IoT and agriculture project on ThingSpeak and Raspberry Pi. Two things in there may still be worth knowing:

- A team member who had been absent for six weeks wanted to **restart the project from scratch**, partly because the GitHub structure was “childish” (single main branch). Scott's answer: at week 7, restarting is not recommended; capture alternatives as documented future work, note the proposed benefits, and put the team's resources behind the agreed plan. Mid-project language or architecture rewrites show up as a regression bug spike.
- On tender documents drifting: *“The tender doc is a snapshot in time, we expect things to change, that's normal.”*